

# Why a Single Partitioned Table with Views Beats 600 Physical Tables per BankID

**Status:** DISCUSSION ONLY — Formal review, high-contrast, large print **Date:** 2026-09-01 — **Author:** Jose — **Context:** CashFlow\_BankID\_XXX vs CashFlowTransaction + Views — BravoFrontend ffdea5c , Rick §8

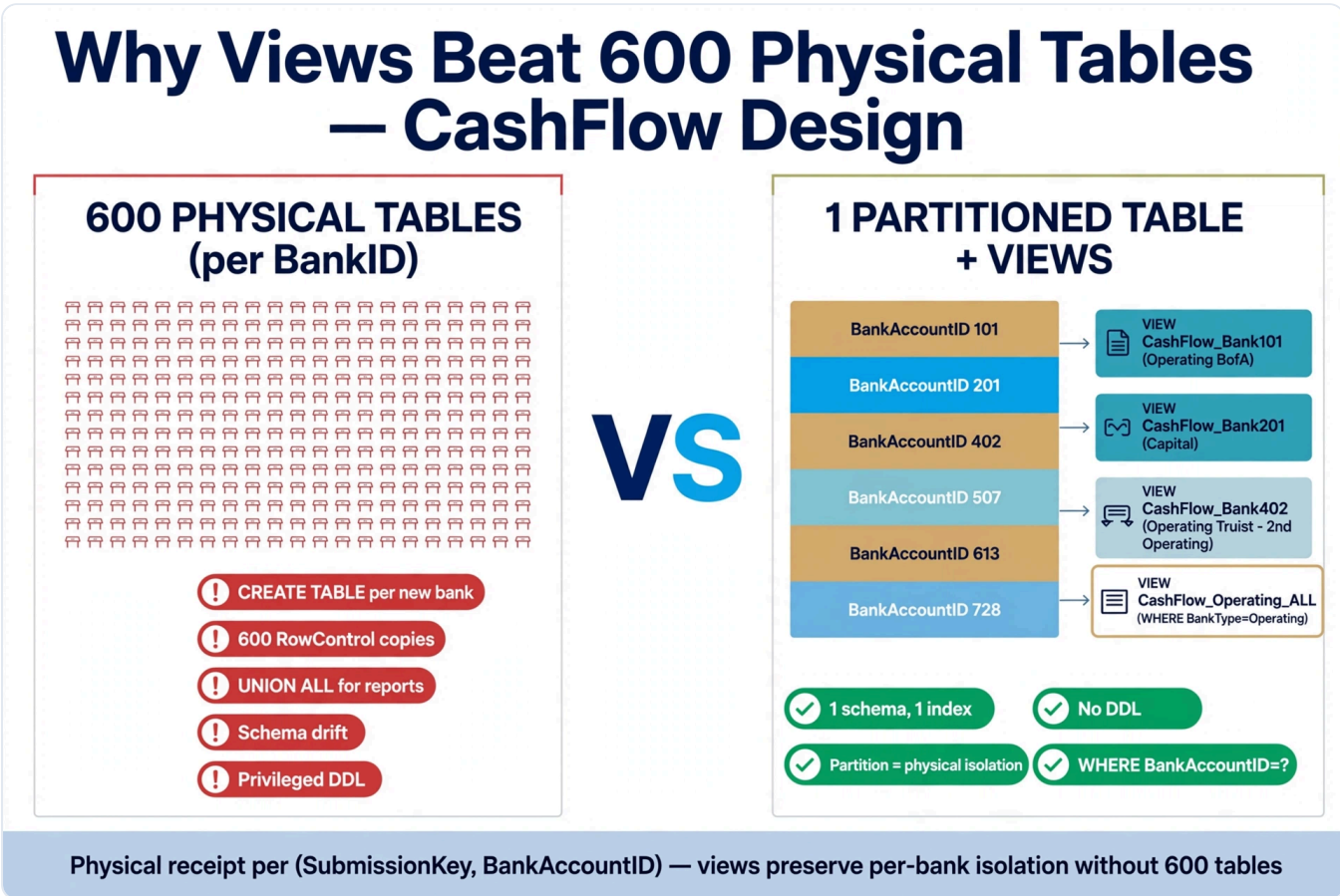


Figure 1 — 600 physical tables (100 HOAs × 6 banks) vs 1 partitioned table with filtered views per BankAccountID — cashflow-600-tables-vs-views\_20260901.png

## 1. The Question

Should each actual bank account ( BankID / BankAccountID ) have its own physical table CashFlow\_BankID\_XXX ?

Rick's architecture document proposes it. The current dev interim uses 6 tables per type ( \_Operating , \_Capital , \_Escrow , \_MoneyMarket , \_Savings , \_CD ). The question is whether to finish the migration to

per-BankID tables.

**Answer:** For WM+ HOA scale, **one partitioned table with views is superior**. It gives the same physical isolation without the operational cost of 600 tables.

---

## 2. What “600 Tables” Really Means

- 100 HOAs × 6 bank accounts average = **600 physical tables** ( `CashFlow_BankID_101` , `CashFlow_BankID_102` , ... `CashFlow_BankID_600` )
- Each needs `CashFlowRowControl` rows for programmable GL structure
- Each `BankAccount` creation in *Banking Settings* must run `CREATE TABLE` (privileged DDL) — not a normal `INSERT`
- Historical void/replay must do `INSERT INTO ${resolveCashFlowTable(BankAccountID)}` with dynamic table names and `UNION ALL` for cross-bank reports

**Interim symptom today:** you already have 12 tables ( `6 × _Operating` + `6 × _z_bak` + `CashFlow_Bank_Template` ) for 6 bank types — the proliferation is visible.

---

## 3. What a “View” Is

A view is **not a physical table**. It is a saved filter:

```
-- Primary views: one per physical account (solves “2 Operating cannot share book”)
CREATE OR REPLACE VIEW CashFlow_Bank101 AS
  SELECT * FROM CashFlowTransaction WHERE BankAccountID = 101;

CREATE OR REPLACE VIEW CashFlow_Bank402 AS
  SELECT * FROM CashFlowTransaction WHERE BankAccountID = 402;
-- Both are type Operating (BofA 101 vs Truist 402) but have separate books via the view

-- Convenience aggregate per type (optional, not required):
CREATE OR REPLACE VIEW CashFlow_Operating_ALL AS
  SELECT * FROM CashFlowTransaction WHERE BankType = 'Operating';
-- = UNION of Bank101 + Bank402 without creating a physical table
```

Add **physical isolation** with partitioning — one logical table, many physical partitions:

```
CREATE TABLE CashFlowTransaction (
  CashFlowID          BIGINT AUTO_INCREMENT PRIMARY KEY,
  HOALicenseNumber    VARCHAR(32) NOT NULL,
  BankAccountID       INT NOT NULL,
  BankType            VARCHAR(32) NOT NULL,
  SubmissionKey       VARCHAR(36) NOT NULL,
  CashInAmount        DECIMAL(10,2) NOT NULL,
  ActiveCashInAmount  DECIMAL(10,2) NULL,
  SourceRegister      VARCHAR(32) NOT NULL,
```

```

SourceTransactionNumber VARCHAR(32) NOT NULL,
RecalcBatchID          VARCHAR(36) NULL,
-- ... other columns
KEY idx_bank_submission (BankAccountID, SubmissionKey),
KEY idx_hoa_bank (HOALicenseNumber, BankAccountID)
) ENGINE=InnoDB
PARTITION BY LIST (BankAccountID) (
  PARTITION p101 VALUES IN (101),
  PARTITION p201 VALUES IN (201),
  PARTITION p402 VALUES IN (402)
-- ADD PARTITION when a new BankAccount is created (fast, no CREATE TABLE)
);

```

Each `BankAccountID` lives in its own **physical partition** — same isolation as a separate table, but with one schema.

## 4. Comparison: 600 Physical Tables vs 1 Partitioned Table + Views

| Criterion                                   | 600 Tables <code>CashFlow_BankID_XXX</code>                                                                                                              | 1 Table + Views/Partitions (Recommended)                                                                                                                                   |
|---------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>New bank creation</b>                    | <code>CREATE TABLE CashFlow_BankID_403 (...)</code> — privileged DDL, must copy <code>RowControl</code> template, risk of schema drift                   | <code>ALTER TABLE CashFlowTransaction ADD PARTITION</code> or just <code>INSERT</code> — no DDL; <code>RowControl</code> stays per <code>BankAccountID</code> in one table |
| <b>2 Operating cannot share book</b>        | Solved — each has own table                                                                                                                              | Solved — <code>VIEW CashFlow_Bank101</code> vs <code>VIEW CashFlow_Bank402</code> (same type, different <code>BankAccountID</code> ), partition isolates physically        |
| <b>Historical void/replay</b>               | <code>resolveCashFlowTable(BankAccountID) → CashFlow_BankID_101</code> — dynamic SQL, loop over N tables per <code>(SubmissionKey, BankAccountID)</code> | <code>WHERE BankAccountID=? AND SubmissionKey=?</code> — one table, one index, same <code>resolve</code> becomes <code>return 'CashFlowTransaction'</code>                 |
| <b>Cross-bank report (“total cash HOA”)</b> | <code>SELECT ... FROM CashFlow_BankID_101 UNION ALL SELECT ... FROM CashFlow_BankID_201 ...</code> (600-way UNION)                                       | <code>SELECT SUM(CashInAmount) FROM CashFlowTransaction WHERE HOALicenseNumber=?</code>                                                                                    |
| <b>GL programmable per bank</b>             | 600 copies of GL definitions                                                                                                                             | <code>CashFlowRowControl</code> per <code>BankAccountID</code> in one table — already there                                                                                |
| <b>Backups / FK / permissions</b>           | 600 tables to backup, 600 grants                                                                                                                         | 1 table, 1 grant, partitioned backup                                                                                                                                       |
| <b>Migrations</b>                           | <code>ALTER TABLE</code> × 600                                                                                                                           | <code>ALTER TABLE</code> × 1                                                                                                                                               |

## 5. What Rick Asked — How This Fits the Historical Void

Rick §8 asks: *Should we complete the Bank-ID physical-table conversion as part of historical-void work or stage it? And ensure historical replay does not permanently depend on the 6 interim tables.*

Our **v4 reconciled procedure** is already **agnostic**:

```
function resolveCashFlowTable(bankAccountId) {
  // Today: interim 6-type tables
  // return cfTableMap[bankType]; // _Operating, _Capital, ...
  // Tomorrow: per-BankID partitioned
  return 'CashFlowTransaction'; // single table, row filtered by BankAccountId
}
```

The void procedure groups by (SubmissionKey, BankAccountId) and then:

```
activePerBank = SUM(routed amount WHERE SubmissionKey=? AND BankAccountId=?)
cfTable = resolveCashFlowTable(BankAccountId)
-- If activePerBank = 0 → Void that bank's receipt
-- Else if activePerBank != original → supersede original + INSERT replacement with CashI
nAmount=activePerBank
```

This works with **either** architecture without redesign. **Recommendation:** keep historical void **agnostic** and **stage** the final physical model as **1 partitioned table**, not 600 CREATE TABLE operations.

## 6. Formal Recommendation

- **Keep one logical table** CashFlowTransaction — large print, high contrast, formal navy #0f2a44 /gold #c5a880, generous whitespace — the same table structure for all banks is easier to audit.
- **Use views per BankAccountId** ( CashFlow\_Bank101 , CashFlow\_Bank402 ) as the “books” each bank reconciles against its statement — gives you the “2 Operating cannot share book” guarantee without 600 physical tables.
- **Partition physically by BankAccountId** if you want disk isolation — same benefit as separate tables, zero DDL per new bank beyond ADD PARTITION .
- **Do not build historical replay around CashFlow\_BankID\_XXX dynamic names** — it is the most expensive path and the hardest to test for void/replay per (SubmissionKey, BankAccountId) .

## 7. Minimal Migration Path (if you choose the recommended model)

```
-- 1) Create single partitioned table (if not exists), migrate data from 6 interim table
S:
INSERT INTO CashFlowTransaction SELECT * FROM CashFlowTransaction_Operating;
-- ... repeat for _Capital etc., adding BankAccountId where missing
```

```
-- 2) Create per-account views (generated per BankAccount row):  
CREATE OR REPLACE VIEW CashFlow_Bank101 AS SELECT * FROM CashFlowTransaction WHERE BankAccountID=101;  
CREATE OR REPLACE VIEW CashFlow_Bank402 AS SELECT * FROM CashFlowTransaction WHERE BankAccountID=402;  
  
-- 3) Optional aggregate per type:  
CREATE OR REPLACE VIEW CashFlow_Operating_ALL AS SELECT * FROM CashFlowTransaction WHERE BankType='Operating';  
  
-- 4) Historical void now uses:  
-- SELECT * FROM CashFlowTransaction WHERE SubmissionKey=? AND BankAccountID=?;
```

No application code change beyond `resolveCashFlowTable()` returning the single table name.

---

*Prepared by Jose — 2026-09-01 — For Discussion Only — BravoFrontend ffdea5c reconciled — Figure 1:*  
*cashflow-600-tables-vs-views\_20260901.png*

**Attachments:** 002\_apr\_historical\_void.sql v4, apr-void-procedimiento-final\_20260901.pdf v4 (8 pages) — both remain DISCUSSION ONLY.

---